Benchmarking AI Factories on MeluXina

Project Report

EUMaster4HPC Student Challenge 2025-2026

Team 11

24.02.2026

Abstract

This project report documents the work carried out by Team 11 as part of the EUMaster4HPC Student Challenge 2025-2026. The objective was to design and implement a modular benchmarking framework for AI workloads on MeluXina; technical design details are presented in the accompanying technical report. Here we describe the results achieved, team organisation, project timeline, and lessons learned.

Contents

1 Project Overview and Evaluation **1**
1.1 Context and Objectives . . . 1
1.2 Work Delivered . . . 2
1.3 Evaluation Methodology and Results . . . 2

2 Organisation **2**
2.1 Team Composition . . . 2
2.2 Work Calendar . . . 3
2.3 Collaboration Practices . . . 3

3 Lessons Learned **4**
3.1 Technical Growth and Exposure to Real-World HPC Infrastructure . . . 4
3.2 Learning to Design Modular, Reproducible Systems . . . 4
3.3 Working in a Group: Coordination, Communication, and Shared Ownership . . . 4
3.4 Broader Perspective on AI and HPC Convergence . . . 5

1 Project Overview and Evaluation

1.1 Context and Objectives

The work presented in this report was carried out in the context of the EUMaster4HPC Student Challenge and focuses on the benchmarking of AI-related workloads on high-performance computing (HPC) systems.

The main objective of this project was to design and implement a modular benchmarking framework capable of evaluating AI workloads on the MeluXina supercomputer. The framework was developed with an emphasis on reproducibility, flexibility, and scalability, while remaining compatible with HPC-specific constraints, including batch scheduling, containerized execution, and multi-node deployments. In addition, the project aimed to produce performance indicators that enable analysis and comparison across different benchmark configurations.

1.2 Work Delivered

Over the duration of the project, our team (Luca, Emanuele, Thomas, and Patrick) developed a Python-based benchmarking framework designed to orchestrate, execute, and monitor AI workloads in an HPC environment. The framework supports the deployment and evaluation of multiple service types, with focus on large language model inference servers and object storage systems.

The delivered solution follows a modular architecture composed of servers, clients, monitors, and loggers. Server components represent long-running services such as vLLM inference servers or MinIO object storage instances. Client components generate configurable workloads that interact with these services in order to measure performance under load. Monitoring components collect system- and application-level metrics through Prometheus-compatible endpoints, while logging components ensure consistent and thread-safe collection of execution logs.

The framework integrates natively with the MeluXina software environment by relying on SLURM for job submission and resource management, and Apptainer for containerized execution. Benchmark experiments are fully described through declarative YAML configuration files, referred to as *recipes*, which specify the services to deploy, the workloads to execute, and the monitoring configuration. This approach enables reproducible experimentation and simplifies adaptation to new benchmark scenarios without modifying the core implementation.

1.3 Evaluation Methodology and Results

The evaluation of the delivered work was conducted through a series of benchmark experiments executed on the MeluXina supercomputer. Two representative use cases were selected: inference benchmarking using a vLLM-based language model server, and storage benchmarking using a MinIO S3-compatible object storage service. These use cases were chosen to cover both compute-intensive and data-intensive components commonly found in AI Factory environments.

Performance evaluation relied on a combination of workload-level metrics and service-level monitoring data. For inference workloads, key indicators included request throughput, response latency, success rate, and concurrency levels. For storage workloads, metrics such as request rate, data transfer volume, and latency distributions were collected. Metrics were continuously gathered during benchmark execution using Prometheus-based monitoring and stored in structured formats to support post-processing and visualization.

The experimental results demonstrate that the framework is able to reliably deploy services, generate controlled workloads, and collect consistent performance data in an HPC environment. The modular design also proved effective in supporting multiple benchmark scenarios with minimal configuration changes. Overall, the evaluation confirms that the developed framework meets its intended goals as a flexible and reproducible benchmarking tool for AI workloads on large-scale HPC systems.

2 Organisation

This section describes the team structure, project timeline, and collaboration practices adopted throughout the challenge. The phased structure of the challenge and the weekly mentoring sessions ensured that all team members remained informed of the project's status and could contribute within their areas of expertise.

2.1 Team Composition

Team 11 consisted of four members from the EUMaster4HPC programme, each bringing complementary skills and backgrounds (Table 1).

Table 1: Team 11 members and responsibilities

Member	**Role**	**Key Contributions**
Luca Lamperti	Lead Developer	Core framework implementation, recipe system, containerisation
Thomas Gantz	Architecture & Documentation	System architecture design, client-server prototyping, project reporting
Patrick Cerka	Prototyping & Infrastructure	Executor module prototyping, MeluXina environment setup, SLURM scripting
Emmanuele Lovino	Testing & Validation	MeluXina onboarding, runtime testing, workload validation

The project was overseen by Dr. Farouk Mansouri from LuxProvide, who provided strategic oversight. Regular weekly technical mentoring sessions were conducted by Mario Serrano Sanchez (LuxProvide), offering structured progress reviews and targeted technical support.

Work was distributed according to individual strengths and availability, following the natural decomposition into the four modules mandated by the challenge brief (Server, Client, Monitor, Logger). The modular architecture provided a clear framework for dividing responsibilities.

2.2 Work Calendar

The challenge spanned four months (October 2025 to January 2026). Table 2 summarises the timeline and mapping to deliverables.

Table 2: Project calendar

Phase	**Period**	**Activities**
1 - Onboarding	Oct 2025	MeluXina access setup; first SLURM jobs (each member ran hello-world and Ollama examples); familiarisation with Apptainer, scratch storage, and GPU partitions; initial architecture design (UML diagram); GitHub repository creation with milestones and issues
2 - Prototyping	Nov 2025	Implementation of abstract base classes (`Executor`, `Monitor`, `Logger`); concrete executors (`SlurmExecutor`, `ProcessExecutor`, `ApptainerExecutor`, `WorkloadExecutor`); `BenchmarkManager` and CLI interface; first recipe definitions; iterative debugging on MeluXina
3 - Evaluation	Dec 2025	vLLM inference benchmark execution; MinIO S3 upload benchmark; Prometheus monitoring and Grafana dashboard integration; `FastAPI` datasource server; automation script for local-to-cluster workflow; performance data collection and analysis
4 - Reporting	Jan 2026	Technical and project report writing; README finalisation with demo recordings

2.3 Collaboration Practices

The team adopted the following tools and practices for coordination:

- **GitHub:** Public repository (`LucaLM02/Benchmarking-AI-Factories`) used for version control, issue tracking, and milestone management.
- **Meetings:** Regular check-ins within the team to discuss progress, blockers, and upcoming tasks, complemented by the weekly mentoring sessions.
- **Issue-driven workflow:** Tasks were tracked as GitHub issues aligned with the challenge phases, allowing transparent progress monitoring.
- **Shared MeluXina workspace:** All team members had access to the project scratch directory on MeluXina, enabling collaborative testing and log sharing.

3 Lessons Learned

3.1 Technical Growth and Exposure to Real-World HPC Infrastructure

One of the most valuable aspects of this challenge was the chance to work on an actual production supercomputer rather than the usual controlled academic setting. Using the MeluXina system meant dealing with the same constraints faced in real HPC environments: submitting jobs through Slurm, working with Apptainer instead of Docker, understanding unfamiliar filesystem layouts, and managing GPU partitions. At times this was frustrating, especially at the beginning, but it forced us to slow down, read documentation carefully, and understand how these systems are meant to be used. That process made the learning much more concrete than typical coursework.

The project also helped us understand what building an "AI Factory" really involves. It is not just about running a model, but about connecting multiple components into a working pipeline: storing and accessing data through object storage, serving models efficiently, and collecting metrics to understand system behavior. By designing benchmarks that exercised each of these pieces and tying them together into an automated workflow, we started to see the full picture of how AI infrastructure operates in practice.

3.2 Learning to Design Modular, Reproducible Systems

Another important lesson was the need for modularity and reproducibility. Early in the project we often wrote quick scripts just to test an idea or get a result. As the work progressed, it became clear that benchmarking is never a one-shot task: parameters change, experiments must be repeated, and results need to be comparable over time. This pushed us to reorganize the code into clearer components, separating execution logic, monitoring, logging, and workload definitions.

Using YAML configuration files to describe experiments turned out to be especially useful. It allowed us to modify setups without rewriting code and made it easier to rerun tests in a consistent way. Automating the full pipeline (from syncing code to the cluster, to launching jobs, to collecting outputs) also reduced errors and saved time. Building the `automation_script.sh` entry point encouraged us to think about usability and reproducibility from the perspective of someone who did not write the original code.

3.3 Working in a Group: Coordination, Communication, and Shared Ownership

Collaborating as a team added another layer of learning. Each member brought different assumptions and experience, so decisions about architecture, metrics, and implementation had to be worked out together rather than taken for granted.

We also had to establish practical habits: using Git consistently, documenting decisions, and dividing tasks clearly. Writing clear commit messages, reviewing each other's work, and keeping documentation updated may seem minor, but they made a big difference as the project grew.

These are the kinds of practices that are easy to overlook in small assignments but become essential in larger efforts.

Of course, group work was not always smooth. Waiting for access to resources, debugging issues introduced by changes, or reconciling different coding styles sometimes slowed us down. Learning to handle these moments constructively by communicating clearly and focusing on the shared objective was an important part of the experience.

3.4 Broader Perspective on AI and HPC Convergence

Finally, the challenge gave us a clearer view of how closely AI and HPC are becoming linked. Systems designed to train, serve, and evaluate models at scale are increasingly treated as core infrastructure. Working on a concrete problem in this space made that trend even clearer and showed us that performance, monitoring, and reproducibility are just as important as the models themselves.

Overall, the challenge acted as an accelerator for both technical and collaborative skills. It required us to engage with real tools, think about systems rather than isolated programs, and deliver something that functions beyond the scope of a typical class project. The experience has better prepared us for complex, multi-disciplinary work in the future.